

TRD 2024

Data Structure

- Min Stack/Queue
- Fenwick Tree
- Segment Tree

DP

- Basic (recursive vs. iterative)
- Bitmask
- Div&Con Optimization
- Knuth Optimization

Graphs

- Traversal (Search, CC, SCC)
- Bridges
- Articulation Point
- Cyclicity
- SSSP
- MST
- Topological Sort
- LCA
- Flows

Strings

- Prefixes KMP

Maths hints

- Combinatorics
- Geometry
- Number theory

Python conventions

By

David Y, Sam H, Yana Y

Swarthmore College

Min Stack/Queue

problem

find min during pushing/popping and queries in $O(1)$ by tracking min on the top of the array

solution

```
///// min stack
stack<pair<int, int>> st;
//adding
int new_min = st.empty() ? new_elem : min(new_elem, st.top().second);
st.push({new_elem, new_min});
//removing
st.pop();
//curr min
st.top().second;
```

```
///// min queue
deque<int> q;
//adding
while (!q.empty() && q.back() > new_element)
    q.pop_back();
q.push_back(new_element);
//removing
if (!q.empty() && q.front() == remove_element)
    q.pop_front();
//curr min
q.front();
```

problem

find minimum in all subarrays of a fixed length in $O(n)$

solution

Use min queue, load $a[0..j]$ to queue, check curr min
then remove $a[0]$, add $a[j+1]$, check curr min

Fenwick Tree

problem

For range sum query $f([a_i, a_{i+1}, \dots, a_j])$, with point updates (better than prefix sum).

- Memory $O(n)$
- Construction $O(n \log(n))$
- Point Update $O(\log(n))$
- Range Query $O(\log(n))$

solution

```
def add(index, value):
    ## Point Update
    while index <= size:
        tree[index] += value
        index += index & -index

def sum(index):
    ## Calculate sum[0,r]
    result = 0
    while index > 0:
        result += self.tree[index]
        index -= index & -index # Jump to the parent node
    return result

global size = MAXN
global tree = [0] * (size + 1)
# use 1-based indexing
add(ind, v_i) # initialize/update for ind= 1...MAXN
print(sum(j)-sum(i-1)) # answer sum[i..j]
```

Segment Tree

problem 1

For associative range query $f([a_i, a_{i+1}, \dots, a_j])$ with point updates.

- Memory $O(4n)$
- Construction $O(n \log(n))$
- Point Update $O(\log(n))$
- Range Query $O(\log(n))$

solution for range sum

```
int n, t[4*MAXN];
// construction
void build(int a[], int v, int tl, int tr) {
    if (tl == tr) {
        t[v] = a[tl];
    } else {
        int tm = (tl + tr) / 2;
        build(a, v*2, tl, tm);
        build(a, v*2+1, tm+1, tr);
        t[v] = t[v*2] + t[v*2+1];
    }
}
// answer query
int lookup(int v, int tl, int tr, int l, int r) {
    if (l > r)
        return 0;
    if (l == tl && r == tr) {
        return t[v];
    }
    int tm = (tl + tr) / 2;
    return sum(v*2, tl, tm, l, min(r, tm))
        + sum(v*2+1, tm+1, tr, max(l, tm+1), r);
}
// point update
void update(int v, int tl, int tr, int pos, int new_val) {
    if (tl == tr) {
        t[v] = new_val;
    } else {
        int tm = (tl + tr) / 2;
        if (pos <= tm)
            update(v*2, tl, tm, pos, new_val);
        else
```

```
            update(v*2+1, tm+1, tr, pos, new_val);
        t[v] = t[v*2] + t[v*2+1];
    }
}

void main(){
    build(input_arr, v=1, tl=0, tr=n-1)
    update(v=1, tl=0, tr=n-1, pos, new_val)
    lookup(v=1, tl=0, tr=n-1, l, r)
}
```

solution for other range queries

```
// change nodes of t so it stores the result of desired operations
// change build():
// t[v] = t[v*2] + t[v*2+1] --> t[v] = f(t[v*2] + t[v*2+1]);
// i.e. f= max(), min(), gcd(), lcm()
// change update(), lookup() similarly
```

problem 2

Finding the maximum and the number of times it appears in ranged queries. Essentially a layered operation (max() and max_count addition)

solution

```
// each node stores (max, count)
pair<int, int> t[4*MAXN];
// similar construction with additional layer via combine()
void build(int a[], int v, int tl, int tr) {
    if (tl == tr) {
        t[v] = make_pair(a[tl], 1);
    } else {
        int tm = (tl + tr) / 2;
        build(a, v*2, tl, tm);
        build(a, v*2+1, tm+1, tr);
        t[v] = combine(t[v*2], t[v*2+1]); //here
    }
}
pair<int, int> combine(pair<int, int> a, pair<int, int> b) {
    if (a.first > b.first)
        return a;
    if (b.first > a.first)
        return b;
```

```

        return make_pair(a.first, a.second + b.second);
    }
    pair<int, int> get_max(int v, int tl, int tr, int l, int r) {
        if (l > r)
            return make_pair(-INF, 0);
        if (l == tl && r == tr)
            return t[v];
        int tm = (tl + tr) / 2;
        return combine(get_max(v*2, tl, tm, l, min(r, tm)),
                       get_max(v*2+1, tm+1, tr, max(l, tm+1), r));
    }
    void update(int v, int tl, int tr, int pos, int new_val) {
        if (tl == tr) {
            t[v] = make_pair(new_val, 1);
        } else {
            int tm = (tl + tr) / 2;
            if (pos <= tm)
                update(v*2, tl, tm, pos, new_val);
            else
                update(v*2+1, tm+1, tr, pos, new_val);
            t[v] = combine(t[v*2], t[v*2+1]);
        }
    }
}

```

problem 3

Count number of value x, and search for the k-th x

solution

```

// t would store count of 0 similar to problem2 (without max),
int t[4*MAXN]; //unchanged
void build(), update() //unchanged

// with a different lookup logic:
int find_kth(int v, int tl, int tr, int k) {
    if (k > t[v])
        return -1;
    if (tl == tr)
        return tl;
    int tm = (tl + tr) / 2;
    if (t[v*2] >= k)
        return find_kth(v*2, tl, tm, k);
    else

```

```

        return find_kth(v*2+1, tm+1, tr, k - t[v*2]);
    }
}

```

problem 4

Searching for the first element greater than a given amount

solution

```

// can be solved with max prefix queries in  $O(\log^2 n)$ 
// with another operator, can solve in  $O(\log n)$ 
int get_first(int v, int tl, int tr, int l, int r, int x) {
    if (tl > r || tr < l) return -1;
    if (t[v] <= x) return -1;

    if (tl == tr) return tl;

    int tm = tl + (tr - tl) / 2;
    int left = get_first(2*v, tl, tm, l, r, x);
    if (left != -1) return left;
    return get_first(2*v+1, tm+1, tr, l, r, x);
}

```

problem 5

Lazy Segment Tree: range update (modification on segment of contiguous elements) rather than point update; EXAMPLE: sum query on range adding

solution

```

//SEE MORE ON JRT 2022 LAZY SEG
void build(int a[], int v, int tl, int tr) {
    if (tl == tr) {
        t[v] = a[tl];
    } else {
        int tm = (tl + tr) / 2;
        build(a, v*2, tl, tm);
        build(a, v*2+1, tm+1, tr);
        //STORING ONLY ADDED UPDATE TO PARENT TREE (non-single node)
        t[v] = 0;
    }
}

void update(int v, int tl, int tr, int l, int r, int add) {
    if (l > r)

```

```

    return;
    if (l == tl && r == tr) {
        t[v] += add;
    } else {
        int tm = (tl + tr) / 2;
        update(v*2, tl, tm, l, min(r, tm), add);
        update(v*2+1, tm+1, tr, max(l, tm+1), r, add);
    }
}

int get(int v, int tl, int tr, int pos) {
    if (tl == tr)
        return t[v];
    int tm = (tl + tr) / 2;
    if (pos <= tm)
        return t[v] + get(v*2, tl, tm, pos);
    else
        return t[v] + get(v*2+1, tm+1, tr, pos);
}

```

Basic DP

Recursive vs. Iterative

```
dp = [[None]* i for _ in range(j)]
```

recursive

```

def solve(statei, statej):
    if dp[i][j] != None:
        return dp[i][j]
    if starting_case:
        dp[i][j] = starting_val
        return dp[i][j]

    dp[i][j] = transition(dp[i_del][j_del],...)
    return dp[i][j]

```

```

def main():
    print(solve(finali, finalj))

```

iterative

```

def solve(statei, statej):
    for i in range(statei, endstatei):
        for j in range(statej, endstatej):
            dp[i][j] = transition(dp[i_del][j_del],...)
    return dp[-1][-1]

```

```

def main():
    print(solve(0,0))

```

Bit masks

usage

- finding number of ways to fully fill an area (e.g. chessboard/grid) with some figures (e.g. dominoes)
- finding a way to fill an area with minimum number of figures
- finding a partial fill with minimum number of unfilled space (or cells, in case of grid) or a partial fill with the minimum number of figures, such that no more figures can be added

problem “parquet”

Given a grid of size $N \times M$. Find number of ways to fill the grid with figures of size 2×1 with no cells left unfilled and no overlap.

solution

Define $dp[i, \text{mask}]$: $i=1, \dots, N$, $\text{mask} = 0, \dots, 2^M - 1$. Mask is the state of last row of current grid; j -th bit of mask is 0 if the corresponding cell is filled. We find $dp[N, 0]$ by:

```
def calc(x=0, y=0, mask=0, next_mask=0):
    global n, m, dp
    if x == n:
        return
    if y >= m:
        dp[x+1][next_mask] += dp[x][mask]
    else:
        my_mask = 1 << y
        if mask & my_mask:
            calc(x, y+1, mask, next_mask)
        else:
            calc(x, y+1, mask, next_mask | my_mask)
            if (y+1 < m) and not (mask & my_mask) and
            not (mask & (my_mask << 1)):
                calc(x, y+2, mask, next_mask)

n, m = # Input for n and m
dp = [[0 for _ in range(1 << m)] for _ in range(n+1)] #shifting 1 by m bits
dp[0][0] = 1
for x in range(n):
    for mask in range(1 << m):
        calc(x, 0, mask, 0)
print(dp[n][0])
```

DC Optimization

problem

If $0 \leq i < m, 0 \leq j < n$, and the dp requires:

$$dp(i, j) = \min_{0 \leq k \leq j} dp(i-1, k-1) + C(k, j),$$

and the monotonicity of opt = choice of k for i, j pair is satisfied, $opt(i, j) \leq opt(i, j+1)$, then we use following optimization from $O(mn^2)$ to $O(mn \log n)$:

NOTE: The monotonicity of opt can be shown with special case (Quadrangle inequality): $C(a, c) + C(b, d) \leq C(a, d) + C(b, c)$ for all $a \leq b \leq c \leq d$

solution

```
int m, n;
vector<long long> dp_before, dp_cur;
long long C(int i, int j);
// compute dp_cur[l], ... dp_cur[r] (inclusive)
void compute(int l, int r, int optl, int opttr) {
    if (l > r)
        return;

    int mid = (l + r) >> 1;
    pair<long long, int> best = {LLONG_MAX, -1};

    for (int k = optl; k <= min(mid, opttr); k++) {
        best = min(best, {(k ? dp_before[k-1] : 0) + C(k, mid), k});
    }

    dp_cur[mid] = best.first;
    int opt = best.second;

    compute(l, mid-1, optl, opt);
    compute(mid+1, r, opt, opttr);
}

long long solve() {
    dp_before.assign(n, 0);
    dp_cur.assign(n, 0);

    for (int i = 0; i < n; i++)
        dp_before[i] = C(0, i);

    for (int i = 1; i < m; i++) {
        compute(0, n-1, 0, n-1);
```

```

        dp_before = dp_cur;
    }

    return dp_before[n - 1];
}

```

Knuth Optimization

problem

Similar to the previous problem, $0 \leq i < m, 0 \leq j < n$, but the dp requires:

$$dp(i, j) = \min_{i \leq k \leq j} [dp(i, k) + dp(k + 1, j) + C(i, j)],$$

This transition occurs in: optimal matrix chain multiplication (finding efficient way to multiply matrices together when minimizing number of scalar multiplications); finding optimal partition points to maximize sum among all subarrays.

Optimization requires: $opt(i, j - 1) \leq opt(i, j) \leq opt(i + 1, j)$, can be satisfied with: for all $a \leq b \leq c \leq d$, 1. $C(b, c) \leq C(a, d)$ and 2. $C(a, c) + C(b, d) \leq C(a, d) + C(b, c)$ (quadrangle inequality)

solution

//SEE MORE ON JRT 2022 KNUTH

Traversals

Iterative vs. Recursive

For binary trees: - pre-order (curr-left-right) - in-order (left-curr-right) - post-order (left-right-curr)

In general, for graphs: - DFS go as deep as possible, traverse deeper once meeting the first “nei” node - BFS go as wide as possible, visit all “nei” nodes before traverse deeper DFS can be implemented with a stack or recursively, BFS is implemented with a queue

Traversal Runtime: $O(V + E)$ for adjacency list

```

// recursive DFS
vector<vector<int>> adjList;
vector<bool> visited;
void DFS(int node) {
    visited[node] = true;
    for(int adjNode : adjList[node]) {
        if(!visited[adjNode]) {
            DFS(adjNode);
        }
    }
}

int main() {
    int n, e;
    adjList.resize(n);
    visited.assign(n, false);
    for(int i = 0; i < e; ++i) {
        int u, v;
        cin >> u >> v;
        adjList[u].push_back(v);
        adjList[v].push_back(u);
    }
    DFS(0);
}

```

traversal for CC

Defined for undirected graphs. Can be solved with iterative traversal updating visited array. Number of CC is number of BFS run. Stop when all nodes visited. Runtime: $O(V + E)$

traversal for SCC

Defined for directed graphs. Runtime: $O(V + E)$

//See JRT2022 SCC

Bridges

problem

bridge is an edge that, if removed, disconnects the graph.

solution

Use DFS $O(V + E)$: an edge (v, nei) is a bridge iff nei and its descendants in DFS has no back-edge to v . Solved by keeping time of entry into node `tin` and checking if descendants can come back.

```
int n;
vector<vector<int>> adj;
vector<bool> visited;
vector<int> tin, low;
int timer;
void dfs(int v, int p = -1) { // p back edge to v
    visited[v] = true;
    tin[v] = low[v] = timer++;
    for (int to : adj[v]) {
        if (to == p) continue;
        if (visited[to]) {
            low[v] = min(low[v], tin[to]);
        } else {
            dfs(to, v);
            low[v] = min(low[v], low[to]);
            if (low[to] > tin[v]) // edge(v,to) is a bridge
        }
    }
}
void find_bridges() {
    timer = 0;
    visited.assign(n, false);
    tin.assign(n, -1);
    low.assign(n, -1);
    for (int i = 0; i < n; ++i) {
        if (!visited[i])
            dfs(i);
    }
}
```

Articulation Point

problem

AP are vertices if removed (along with its edges), disconnects graph.

solution

DFS in $O(V + E)$: Start with arbitrary vertex, `root`, observe for vertex v in the DFS: (1) if v is not root, if none of vertices or its descendants has back-edge to any of ancestors of v , v is AP. (2) if v is root, if v has more than one child in the DFS tree, v is AP.

```
/// observe almost identical "time of entry into node" method as Bridge
int n; int timer; vector<vector<int>> adj;
vector<bool> visited; vector<int> tin, low;
void dfs(int v, int p = -1) {
    visited[v] = true;
    tin[v] = low[v] = timer++;
    int children=0;
    for (int to : adj[v]) {
        if (to == p) continue;
        if (visited[to]) {
            low[v] = min(low[v], tin[to]);
        } else {
            dfs(to, v);
            low[v] = min(low[v], low[to]);
            if (low[to] >= tin[v] && p != -1)
                //is AP (case v descendants of root)
                ++children;
        }
    }
    if (p == -1 && children > 1) //is AP (case v==root)
}
void find_cutpoints() {
    timer = 0;
    visited.assign(n, false);
    tin.assign(n, -1);
    low.assign(n, -1);
    for (int i = 0; i < n; ++i) {
        if (!visited[i])
            dfs(i);
    }
}
```

Cyclicity

problem

determine cyclicity.

solution

DFS in $O(E)$: initially label all vertices white (0). From each unvisited (white) vertex, start the DFS, mark it gray (1) while entering and mark it black (2) on exit. If DFS moves to a gray vertex, then we have found a cycle (if the graph is undirected, the edge to parent is not considered).

```
int n;
vector<vector<int>> adj;
vector<bool> visited;
vector<int> parent;
int cycle_start, cycle_end;

bool dfs(int v, int par) { // passing in vertex and its parent vertex
    visited[v] = true;
    for (int u : adj[v]) {
        if(u == par) continue; // skipping edge to parent vertex
        if (visited[u]) {
            cycle_end = v;
            cycle_start = u;
            return true;
        }
        parent[u] = v;
        if (dfs(u, parent[u]))
            return true;
    }
    return false;
}

void find_cycle() {
    visited.assign(n, false);
    parent.assign(n, -1);
    cycle_start = -1;

    for (int v = 0; v < n; v++) {
        if (!visited[v] && dfs(v, parent[v]))
            break;
    }

    if (cycle_start == -1) {
```

```
        cout << "Acyclic" << endl;
    } else {
        vector<int> cycle;
        cycle.push_back(cycle_start);
        for (int v = cycle_end; v != cycle_start; v = parent[v])
            cycle.push_back(v);
        cycle.push_back(cycle_start);

        cout << "Cycle found: ";
        for (int v : cycle)
            cout << v << " ";
        cout << endl;
    }
}
```

SSSP

problem

output shortest path from s to all reachable v

solution

Dense Dijkstra: keeps distance $d[v]=\text{inf}$, $d[s]=0$ as distance to s , keeping marking $u[v]=0$; run n iterations, chooses unmarked closest v : - v gets marked, all v 's neighbor updates $d[to] = \min(d[to], d[v] + \text{len}(v, to))$ - Runtime: $O((V + E) \log V)$

```
const int INF = 1000000000;
vector<vector<pair<int, int>>> adj;
void dijkstra(int s, vector<int> & d, vector<int> & p) {
    int n = adj.size();
    d.assign(n, INF);
    p.assign(n, -1);
    vector<bool> u(n, false);
    d[s] = 0;
    for (int i = 0; i < n; i++) {
        int v = -1;
        for (int j = 0; j < n; j++) {
            if (!u[j] && (v == -1 || d[j] < d[v]))
                v = j;
        }
        if (d[v] == INF)
            break;
        u[v] = true;
```

```

    for (auto edge : adj[v]) {
        int to = edge.first;
        int len = edge.second;

        if (d[v] + len < d[to]) {
            d[to] = d[v] + len;
            p[to] = v;
        }
    }
}
}

```

solution *

Sparse Dijkstra: - heap supported v selection rather than marking - Runtime: $O((V + E) \log V)$

```

const int INF = 1000000000;
vector<vector<pair<int, int>>> adj;
void dijkstra(int s, vector<int> & d, vector<int> & p) {
    int n = adj.size();
    d.assign(n, INF);
    p.assign(n, -1);

    d[s] = 0;
    using pii = pair<int, int>;
    priority_queue<pii, vector<pii>, greater<pii>> q;
    q.push({0, s});
    while (!q.empty()) {
        int v = q.top().second;
        int d_v = q.top().first;
        q.pop();
        if (d_v != d[v])
            continue;

        for (auto edge : adj[v]) {
            int to = edge.first;
            int len = edge.second;

            if (d[v] + len < d[to]) {
                d[to] = d[v] + len;
                p[to] = v;
                q.push({d[to], to});
            }
        }
    }
}

```

MST

problem

weighted, undirected graph; find one spanning tree to all vertices with least weight

solution 1

Dense tree (connecting n points on cartesian plane): runtime $O(V^2)$

```

int n;
vector<vector<int>> adj; // adjacency matrix of graph
const int INF = 1000000000; // weight INF means there is no edge
struct Edge {
    int w = INF, to = -1;
};
void prim() {
    int total_weight = 0;
    vector<bool> selected(n, false);
    vector<Edge> min_e(n);
    min_e[0].w = 0;

    for (int i=0; i<n; ++i) {
        int v = -1;
        for (int j = 0; j < n; ++j) {
            if (!selected[j] && (v == -1 || min_e[j].w < min_e[v].w))
                v = j;
        }
        if (min_e[v].w == INF) {
            exit(0); // no MST exists
        }
        selected[v] = true;
        total_weight += min_e[v].w;
        if (min_e[v].to != -1)
            cout << v << " " << min_e[v].to << endl;
        for (int to = 0; to < n; ++to) {
            if (adj[v][to] < min_e[to].w)
                min_e[to] = {adj[v][to], v};
        }
    }
    cout << total_weight << endl;
}

```

solution 2

Sparse Tree (tree-esque): runtime $O(m \log n)$ (python heap implementable)

```
int n; vector<vector<Edge>> adj; const int INF = 1000000000;
struct Edge {
    int w = INF, to = -1;
    bool operator<(Edge const& other) const {
        return make_pair(w, to) < make_pair(other.w, other.to);
    }
};
void prim() {
    int total_weight = 0;
    vector<Edge> min_e(n);
    min_e[0].w = 0;
    set<Edge> q;
    q.insert({0, 0});
    vector<bool> selected(n, false);
    for (int i = 0; i < n; ++i) {
        if (q.empty()) {
            exit(0); // no MST
        }
        int v = q.begin()->to;
        selected[v] = true;
        total_weight += q.begin()->w;
        q.erase(q.begin());
        if (min_e[v].to != -1)
            cout << v << " " << min_e[v].to << endl;
        for (Edge e : adj[v]) {
            if (!selected[e.to] && e.w < min_e[e.to].w) {
                q.erase({min_e[e.to].w, e.to});
                min_e[e.to] = {e.w, v};
                q.insert({e.w, e.to});
            }
        }
    }
    cout << total_weight << endl;}
```

Topological Sort

problem

In directed graph, find permutation of vertices corresponds to the order defined by all edges of the graph. General topological order problem: n variables with unknown values, for some variables we know one is less than the other, find contradiction or output value in ascending order. $\Leftrightarrow$ topological sort graph with n vertices.

solution

```
int n;
vector<vector<int>> adj;
vector<bool> visited;
vector<int> ans;
void dfs(int v) {
    visited[v] = true;
    for (int u : adj[v]) {
        if (!visited[u])
            dfs(u);
    }
    ans.push_back(v);
}
void topological_sort() {
    visited.assign(n, false);
    ans.clear();
    for (int i = 0; i < n; ++i) {
        if (!visited[i]) {
            dfs(i);
        }
    }
    reverse(ans.begin(), ans.end());
}
```

LCA

problem

Given a tree G , and queries $(v1, v2)$, find the lowest common ancestor.

solution (Euler Path)

Preprocess tree before queries by DFS, starting from root. The vertices in traversal has order of visited `euler` and height `height` both size N . The LCA is the node with least height during the euler path $v1$ to $v2$. Becomes a RMQ problem (see data structure); segtree runtime: $O(\log N)$, preprocess: $O(N)$

//Seg tree implementation

```
struct LCA {
    vector<int> height, euler, first, segtree;
    vector<bool> visited;
    int n;
    LCA(vector<vector<int>> &adj, int root = 0) {
        n = adj.size();
        height.resize(n);
        first.resize(n);
        euler.reserve(n * 2);
        visited.assign(n, false);
        dfs(adj, root);
        int m = euler.size();
        segtree.resize(m * 4);
        build(1, 0, m - 1);
    }
    void dfs(vector<vector<int>> &adj, int node, int h = 0) {
        visited[node] = true;
        height[node] = h;
        first[node] = euler.size();
        euler.push_back(node);
        for (auto to : adj[node]) {
            if (!visited[to]) {
                dfs(adj, to, h + 1);
                euler.push_back(node);
            }
        }
    }
    void build(int node, int b, int e) {
        if (b == e) {
            segtree[node] = euler[b];
        } else {
            int mid = (b + e) / 2;
```

```
            build(node << 1, b, mid);
            build(node << 1 | 1, mid + 1, e);
            int l = segtree[node << 1], r = segtree[node << 1 | 1];
            segtree[node] = (height[l] < height[r]) ? l : r;
        }
    }
    int query(int node, int b, int e, int L, int R) {
        if (b > R || e < L)
            return -1;
        if (b >= L && e <= R)
            return segtree[node];
        int mid = (b + e) >> 1;
        int left = query(node << 1, b, mid, L, R);
        int right = query(node << 1 | 1, mid + 1, e, L, R);
        if (left == -1) return right;
        if (right == -1) return left;
        return height[left] < height[right] ? left : right;
    }
    int lca(int u, int v) {
        int left = first[u], right = first[v];
        if (left > right)
            swap(left, right);
        return query(1, 0, euler.size() - 1, left, right);
    }
};
```

solution (Binary Lifting)*

Runtime: $O(\log N)$, preprocess $O(N \log N)$

//Potentially better and easier implementation

//See JRT 2022 LCA

Maximum Flow

Ford Fulkerson

```
def bfs(rGraph, s, t, parent):
    #BFS on residual graph for an augmenting path.
    visited = [False] * len(rGraph)
    queue = deque([s])
    visited[s] = True

    while queue:
        u = queue.popleft()
        for ind, val in enumerate(rGraph[u]):
            if visited[ind] == False and val > 0:
                # If not visited and there's capacity
                queue.append(ind)
                visited[ind] = True
                parent[ind] = u
            if ind == t:
                return True # If sink is reached

    return False

def fordFulkerson(graph, source, sink):
    # This will be our flow network where rGraph[u][v] indicates
    # residual capacity of edge from u to v.
    # If rGraph[u][v] is 0, there's no edge.
    rGraph = [row[:] for row in graph]
    max_flow = 0
    parent = [-1] * len(graph)

    # Augment the flow while there is a path from source to sink
    while bfs(rGraph, source, sink, parent):
        # Find the min resid capacity along the path filled by BFS
        path_flow = float('inf')
        s = sink
        while(s != source):
            path_flow = min(path_flow, rGraph[parent[s]][s])
            s = parent[s]

        # Add path flow to overall flow
        max_flow += path_flow
        # Update residual capacities of the edges
        # reverse edges along the path
        v = sink
```

```
        while(v != source):
            u = parent[v]
            rGraph[u][v] -= path_flow
            rGraph[v][u] += path_flow
            v = parent[v]

    return max_flow
```

Edmonds-Karp

```
int n; vector<vector<int>> capacity; vector<vector<int>> adj;
int bfs(int s, int t, vector<int>& parent) {
    fill(parent.begin(), parent.end(), -1);
    parent[s] = -2;
    queue<pair<int, int>> q;
    q.push({s, INF});

    while (!q.empty()) {
        int cur = q.front().first;
        int flow = q.front().second;
        q.pop();
        for (int next : adj[cur]) {
            if (parent[next] == -1 && capacity[cur][next]) {
                parent[next] = cur;
                int new_flow = min(flow, capacity[cur][next]);
                if (next == t)
                    return new_flow;
                q.push({next, new_flow});
            }
        }
    }
    return 0;
}

int maxflow(int s, int t) {
    int flow = 0;
    vector<int> parent(n);
    int new_flow;
    while (new_flow = bfs(s, t, parent)) {
        flow += new_flow;
        int cur = t;
        while (cur != s) {
            int prev = parent[cur];
            capacity[prev][cur] -= new_flow;
            capacity[cur][prev] += new_flow;
            cur = prev;
        }
    }
    return flow;
}
```

Prefix Function KMP

problem

For a string of size n and its substrings $[0..i]$, find the longest proper prefix of the substring which is also a suffix of the substring.

solution

Runtime: $O(N)$

```
vector<int> prefix_function(string s) {
    int n = (int)s.length();
    vector<int> pi(n);
    for (int i = 1; i < n; i++) {
        int j = pi[i-1];
        while (j > 0 && s[i] != s[j])
            j = pi[j-1];
        if (s[i] == s[j])
            j++;
        pi[i] = j;
    }
    return pi;
}
```

problem 1

Given a text t and string s , find the positions of all occurrences of the string s in the text t .

solution

Runtime: $O(T+N)$ We can solve by generating master string $M = s*+t$, where $*$ is not existent in s , t . Then by use prefix function on M , loop through to find occurrences in pi s.t. $pi[j] == n$, meaning the suffix is also the prefix of length n , which is just our s .

problem 2

Given string s , count the number of appearances of each prefix $s[0..i]$ in the same string.

solution

Observe $pi[i]$, prefix of length $== pi[i]$ occurs and ends at position i . There's no longer prefix that follows and shorter prefixes can end at this position. Thus compute:

```
vector<int> ans(n + 1);
for (int i = 0; i < n; i++)
    ans[pi[i]]++;
for (int i = n-1; i > 0; i--)
    ans[pi[i-1]] += ans[i];
for (int i = 0; i <= n; i++)
    ans[i]++; //+1 for original trivial prefix
```

problem 3

Find the number of different substrings in a string s of size n .

solution

Runtime: $O(n^2)$. We will solve this problem iteratively: Knowing the current number of different substrings, we recompute this count by adding a character to the end.

k is current number of different substrings in s , add character c to the end of s . We take the new string $t = s + c$ and reverse. The max prefix function $(t) = \pi_{max}$ is the length of the longest prefix that appears in s . All prefixes of smaller length appear as well.

Therefore the new count is: $k' = k + (|s| + 1 - \pi_{max})$

Math

Combinatorics problem (Power of factorial divisor)

Given n, k , find the largest x such that $n!$ is divisible by k^x

solution

For k prime, Every k -th element in factorial is divisible by k . Every k^a -th element in factorial is divisible by k^a

```
int fact_pow (int n, int k) {
    int res = 0;
    while (n) {
        n /= k; //floor
        res += n;
    }
    return res;
}
```

If k not prime, factor k to $k_1^{p_1} k_2^{p_2} \dots$, for each k_i , we find the number of times it is present in $n!$, denote it a_i . The solution is $\min[\frac{a_i}{p_i}]$

Inclusion-Exclusion Principle

problem 1

Count how many permutations of number from 0 to 9 exist such that the first element is greater than 1 and the last one is less than 8.

solution

By IEP, answer = $2 * 9! + 2 * 9! - 2 * 2 * 8!$

problem 2

Count sequences of length n exist consisting only 0, 1, 2 such that each number occurs at least once

solution

$A_i = i$ not occur, answer = $3^n - (3 * 2^n - 3 * 1 + 0)$

problem 3

Consider $x_1 + x_2 + \dots + x_d = s$, $0 \leq x \leq b$, count the number of solutions to the equation.

solution

ans = $\sum_{i=0}^d (-1)^i C(d, i) \cdot C(s + d - 1 - (b + 1)i, d - 1)$

Areas of Simple Polygon problem

polygon without self intersection, not necessarily convex

solution

we go through all edges and add trapezoid areas bounded by each edge and x-axis. The area needs to be taken with sign so that the extra area will be reduced.

```
double area(const vector<point>& fig) {
    double res = 0;
    for (unsigned i = 0; i < fig.size(); i++) {
        point p = i ? fig[i - 1] : fig.back();
        point q = fig[i];
        res += (p.x - q.x) * (p.y + q.y);
    }
    return fabs(res) / 2;
}
```

Pick's Theorem

Simple polygon on a lattice. compute area of the polygon.

solution

Compute the area of this polygon through the number of vertices that are lying on the boundary and the number of vertices that lie strictly inside the polygon.

Denote area = S , number of points with integer coordinates lying strictly inside the polygon by I , and the number of points lying on polygon sides by B .

$$S = I + \frac{B}{2} - 1$$

Python math library

```
import math
math.ceil(x), math.floor(x)
math.sqrt(x)
math.factorial(x)
math.gcd(x), math.lcm(x)
math.pow(x,y), math.log(x, [base=y])
math.sin(x), math.cos(x), math.tan(x)
```

Python conventions

- fast I/O

```
sys.stdin.readline().strip()
```

```
sys.stdout.write(+"\n")
```

- recursion stack limit

```
sys.setrecursionlimit(10**6)
```

- string precision/format

```
f"{number:.2f}" for 2 decimal places.
```

- filter

Quickly filter elements from a list based on condition:

```
filtered = filter(lambda x: x % 2 == 0, myList) Filter even numbers
```

- re

```
## may not be accessible
```

```
import re
```

```
def process_text(text):
```

```
    match text:
```

```
        case text if re.match(r'^\d+', text):
```

```
            return "Starts with a number."
```

```
        case text if re.match(r'^\w+', text):
```

```
            return "Starts with a word character."
```

```
        case _:
```

```
            return "Does not start with a number or word character."
```

```
print(process_text("123Hello")) # Starts with a number.
```

- itertools (P,C, groupby)

```
itertools.permutations(iterable, r) for permutations.
```

```
itertools.combinations(iterable, r) for combinations.
```

```
itertools.groupby(iterable, key=None) for grouping adjacent elements in an iterable.
```

```
from itertools import groupby
```

```
from operator import itemgetter
```

```
students = [('John', 'A'), ('Jane', 'B'), ('Dave', 'A'), ('Mike', 'C'), ('E', 'A')]
# Sort the list by grade since groupby() requires the iterable to be sorted
students.sort(key=itemgetter(1))
```

```
# Group students by their grade
```

```
for grade, group in groupby(students, key=itemgetter(1)):
```

```
    print(f"Grade {grade}: {[name for name, _ in group]}")
```

```
'''
```

```
output:
```

```
Grade A: ['John', 'Dave']
```

```
Grade B: ['Jane', 'Emily']
```

```
Grade C: ['Mike']
```

```
'''
```

- operator.itemgetter()

```
myList = [(2, 'banana'), (1, 'apple'), (3, 'orange')]
```

```
myList.sort(key=operator.itemgetter(1))
```